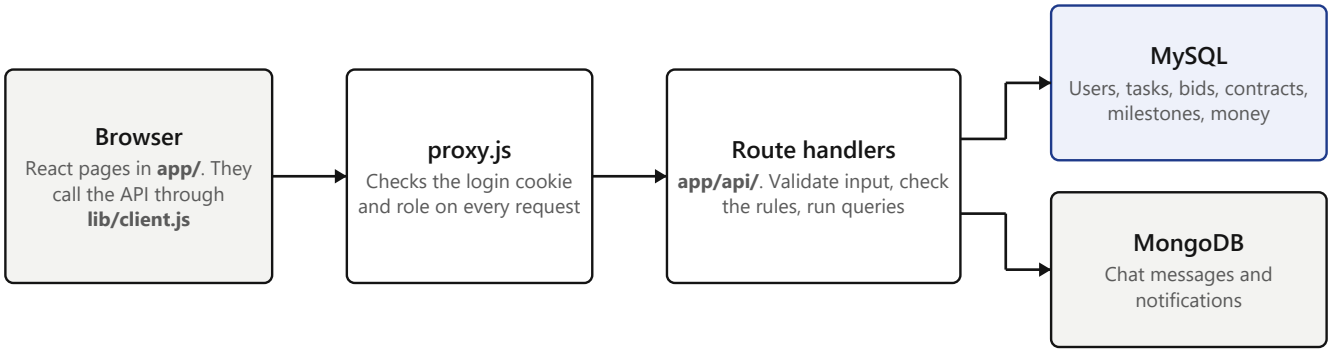
Campus Task Board

Demo and viva guide. Team 4: Aryan Burman (PES2UG24CS088), Atharva Singh (PES2UG24CS095)

> **SAY THIS: THE ONE-MINUTE PITCH**
> Campus Task Board is a marketplace for students at our college. A student posts a small paid task, like a poster design or coding help, and other students bid on it. When the poster accepts a bid, the money is locked in escrow, then released milestone by milestone as work is approved. If something goes wrong, either side can raise a dispute and an admin can refund the poster. The money is simulated. We use MySQL for money and core records because they need transactions and constraints, and MongoDB for chat and notifications.

10	2	2	2	2	2	22	9
tables	views	functions	procedures	triggers	collections	API endpoints	screens

1 How the system fits together



What happens on a click: the page sends a request, `proxy.js` checks who you are, the route handler checks the business rules and runs the SQL, and the page shows the JSON it gets back. Next.js holds the frontend and the API in one project, so there is no separate backend server.

Why two databases?

	MySQL	MongoDB
Holds	Users, tasks, bids, contracts, milestones, disputes, reviews, transaction log	`messages` and `notifications` collections
Why it fits	Money needs transactions, row locks, foreign keys and CHECK constraints	Simple append-and-read documents that need no joins. Losing one is not serious
How they connect	Decides who may read a chat: only the contract's poster and worker	Stores `contract_id` and `user_id` as plain numbers, with no foreign key

Technology

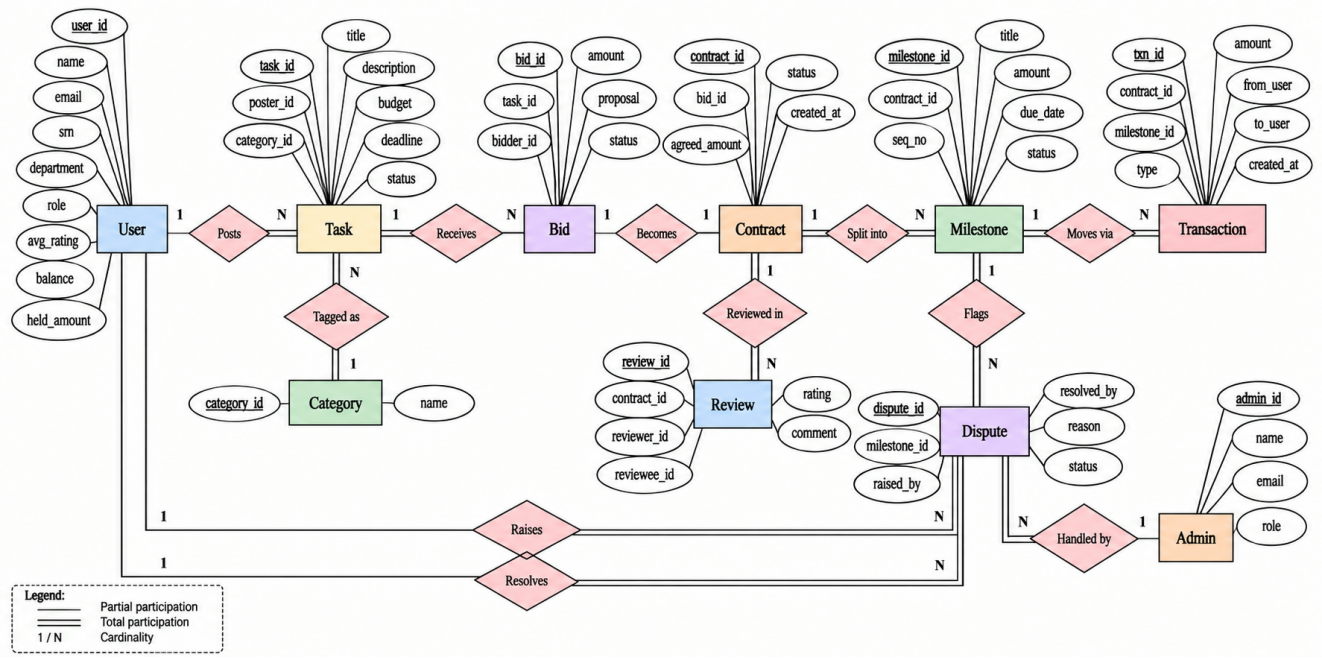
Next.js 16, React 19	Pages and API route handlers
`mysql2` and `mongodb`	Database drivers. MySQL uses a shared pool of up to 10 connections
`bcryptjs`	Stores passwords as salted one-way hashes
`jose`	Signs the login token (JWT, HS256), stored in an httpOnly cookie for 1 day

2 What has been implemented: the workflow

Step	Who	What changes	Money	Database concept to mention
1 Post task	Student	TASK is OPEN	None	CHECK (budget > 0), foreign keys
2 Bid	Other students	BID is PENDING. Poster is notified	None	UNIQUE (task_id, bidder_id): one bid per task
3 Accept bid	Poster	Bid ACCEPTED, other bids REJECTED, task ASSIGNED, CONTRACT created	HOLD	Transaction, row lock, trigger
4 Add milestones	Poster	MILESTONE is PENDING	None	Total cannot exceed the agreed amount, checked inside a transaction
5 Submit	Worker	Milestone SUBMITTED	None	One UPDATE with JOINs does all the checks
6 Approve	Poster	Milestone APPROVED. Once fully settled: contract and task COMPLETED	RELEASE	Transaction, stored procedure
7 Review	Both	REVIEW row added, rating updated	None	Trigger calls a function
Dispute	Poster or worker	Milestone FLAGGED, contract DISPUTED (frozen). Dispute is OPEN	None	Transaction. Money stays in escrow
Refund	Admin	Dispute REFUNDED. Contract returns to ACTIVE, or closes if settled	REFUND	Transaction. The admin list joins 7 tables, using USERS three times

Also built: register and login with roles, browse with search, category and budget filters, a dashboard (wallet, contracts, tasks, bids), chat on each contract, and notifications. A **contract closes** when released plus refunded money reaches the agreed amount: `COMPLETED` if anything was released, `REFUNDED` if everything was refunded (the task then becomes `CANCELLED`).

3 The database



ER diagram. USER and TRANSACTION became USERS and TRANSACTION_LOG in SQL because those names are reserved words.

The 10 tables

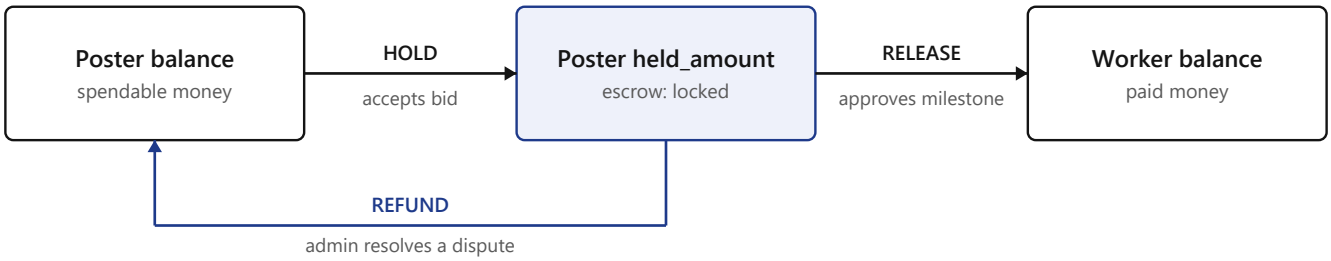
Table	Stores	Key design point
USERS	Students and their wallets	balance and held_amount both have CHECK (>= 0), a last line of defence against overspending
ADMIN	Admins who resolve disputes	Separate table: admins have no wallet, tasks or bids
CATEGORY	Task categories	Kept apart from TASK, so a name is stored once
TASK	Posted work	Status ENUM. category_id may be NULL
BID	Offers on a task	UNIQUE (task_id, bidder_id)
CONTRACT	An accepted bid	bid_id is UNIQUE, which turns 1:N into **1:1**
MILESTONE	Payment stages	Surrogate key, plus UNIQUE (contract_id, seq_no)
REVIEW	Ratings after a contract	UNIQUE (contract_id, reviewer_id), CHECK rating 1 to 5
DISPUTE	Problems raised on a milestone	raised_by points to USERS, admin_id to ADMIN
TRANSACTION_LOG	Audit trail of money moves	milestone_id may be NULL, because the first HOLD covers the whole contract

Views, functions, procedures and triggers

Object	What it does	Where you can see it
vw_user_wallet	Wallet columns only, so no password hash or email	Dashboard wallet
vw_active_contracts	Saves the join of CONTRACT, BID, TASK and USERS (twice) for ACTIVE contracts	Dashboard, Active contracts
fn_avg_rating	Average rating, calculated live from REVIEW	Dashboard, and used by the trigger below
fn_days_left	Days until a task's deadline (negative means overdue)	Browse page and task page
sp_close_task	Sets a task to COMPLETED	Called when a contract completes
sp_get_user_summary	Wallet plus number of bids placed	Dashboard, Bids placed
trg_task_assigned_on_accept	When a bid becomes ACCEPTED, sets the task to ASSIGNED	Accepting a bid
trg_avg_rating_after_review	When a review is added, refreshes the stored avg_rating	Leaving a review

Query demos: alter_and_queries.sql has examples of DELETE with ROLLBACK, aggregates, GROUP BY with HAVING, and scalar, NOT IN, correlated, EXISTS and FROM-clause subqueries.

4 How the money moves: the core of the project



Escrow is not a separate account. It is the `held_amount` column on the poster's own row. Every move also adds a row to `TRANSACTION_LOG`.

Worked example (sample data)	Before	After
Diya accepts Kabir's ₹280 bid	Diya: ₹500 available, ₹0 held	Diya: ₹220 available, ₹280 held
Diya approves a ₹280 milestone	Kabir: ₹300 available	Kabir: ₹580 available. Diya: ₹0 held
Admin refunds Aryan's ₹200 milestone	Aryan: ₹550 available, ₹450 held	Aryan: ₹750 available, ₹250 held

The pattern every money route follows

```
START TRANSACTION
  SELECT ... FROM USERS WHERE user_id = poster FOR UPDATE   -- lock the poster's row
  check the rules: right person? right status? enough balance?
  UPDATE balances, UPDATE statuses, INSERT INTO TRANSACTION_LOG
COMMIT                                                      -- any error means ROLLBACK
notify() in MongoDB                                         -- only after COMMIT
```

ACID, with examples from this project

Atomicity	Accepting a bid changes USERS, BID and TASK and adds CONTRACT and TRANSACTION_LOG rows. All of it commits, or none of it does.
Consistency	CHECK, UNIQUE and foreign keys reject invalid data, so a balance can never go negative.
Isolation	`FOR UPDATE` makes a second request wait. On a double click, the second request sees the milestone is already APPROVED and is refused, so the worker is not paid twice.
Durability	After COMMIT, InnoDB keeps the change even if the server crashes.

5 What each file does

Database and setup

`campus_task_board.sql`	Creates the tables, sample data, views, functions, procedures and triggers, and runs demo queries
`alter_and_queries.sql`	Adds password columns, repairs the sample data for the demo, and holds extra demo queries
`scripts/reset-db.mjs`	Runs for `npm run reset`: rebuilds MySQL and clears MongoDB
`.env.example`	Template for `.env.local`, which holds database settings and the JWT secret
`proxy.js`	Runs before every request. Sends logged-out users to login, and keeps students and admins in their own areas

lib/: shared helpers

`db.js`, `mongo.js`	One shared connection to each database
`session.js`	Signs and verifies the JWT
`auth.js`	Sets the login cookie. `requireStudent()` and `requireAdmin()` guard the routes
`api.js`	`ApiError`, shared error handling and amount validation
`contracts.js`	Loads a contract with its people, checks you belong to it, and closes it when settled
`notify.js`	Saves a notification. A failure is logged but does not fail the request
`client.js`	Browser helper for calling the API and formatting money

app/api/: the 19 route files

Route under /api	Method	Purpose
`auth/register, login, logout, me`	POST, GET (me)	Accounts and sessions. The role is never taken from the form
`tasks`	GET, POST	Browse with filters, or post a task
`tasks/[id]`	GET	One task. The poster sees all bids, others only their own
`tasks/[id]/bids`	POST	Place a bid, and notify the poster
`bids/[id]/accept`	POST	**Transaction.** Hold the money and create the contract
`contracts/[id]`	GET	Everything on the contract page
`contracts/[id]/milestones`	POST	**Transaction.** Add a milestone
`contracts/[id]/messages`	GET, POST	Chat in MongoDB, permission checked in MySQL
`contracts/[id]/review`	POST	Leave a review once the contract is closed
`milestones/[id]/submit`	POST	Worker marks the milestone as done
`milestones/[id]/approve`	POST	**Transaction.** Release the money to the worker
`milestones/[id]/dispute`	POST	**Transaction.** Flag the milestone and freeze the contract
`admin/disputes`	GET	All disputes, open ones first
`admin/disputes/[id]/refund`	POST	**Transaction.** Return the money to the poster
`dashboard`	GET	Uses the views, the procedure and the function
`notifications`	GET, POST	List notifications, or mark them all as read

Pages and components

`app/layout.js`, `app/globals.css`	Shell of every page (fonts, nav bar) and the black-and-white styling
`app/page.js`	Redirects to `/tasks`
`app/login`, `app/register`	Login and registration forms
`app/tasks`	Browse tasks, `new/` to post one, `[id]/` to view, bid and accept
`app/contracts/[id]`	The busiest page: milestones, disputes, chat, transaction log, reviews
`app/dashboard`, `app/notifications`	Student's wallet and activity, and their notifications
`app/admin`	Open and resolved disputes, with the refund button
`components/Nav.js`	Nav bar. Its links depend on the role, and it shows the unread count
`components/ChatBox.js`	Contract chat. Checks for new messages every 3 seconds

6 Demo script

Before you start: start the MySQL and MongoDB services, then run `npm run reset` and `npm run dev`. Open a normal window and an incognito window so two people can be logged in at once, since each window keeps its own login cookie. Every password is `demo1234`. Keep a terminal open for the ROLLBACK step.

Account	Starts with
`aryan@pes.edu`	₹550 available, ₹450 held. Poster on contract 1
`diya@pes.edu`	₹500 available. Worker on contract 1, poster of the open Flask task
`kabir@pes.edu`	₹300 available. Has a pending ₹280 bid on the Flask task
`nandini.admin@pes.edu`	Admin

#	Log in as	Do this	Point out
1	Nobody	Open `/dashboard`, then try a wrong password	proxy.js redirects you. The error does not reveal whether the email exists
2	Aryan	Post a task with budget ₹400	Task is OPEN, and no money is held yet
3	Kabir	Bid ₹350 on it	Bid form disappears, and UNIQUE blocks a second bid in the database too. Aryan gets a MongoDB notification
4	Diya	Open the Flask task and accept Kabir's bid	**Transaction and trigger.** ₹220 available, ₹280 held, HOLD row, task ASSIGNED
5	Diya	Add a ₹300 milestone, then a ₹280 one	The form blocks ₹300 (over the agreed amount), and the API has the same check. ₹280 is added as PENDING
6	Diya, Kabir	Send chat messages from both windows	**MongoDB.** They appear within 3 seconds
7	Kabir, Diya	Kabir submits, Diya approves	**Procedure.** Kabir has ₹580, RELEASE row, contract and task COMPLETED
8	Diya	Leave a 5-star review	**Trigger and function.** Kabir's rating becomes 5.00
9	Aryan	Contract 1: dispute "Draft poster"	Milestone FLAGGED, contract DISPUTED, no money moved
10	Admin	Refund the poster	Aryan: ₹750 and ₹250. REFUND row. Contract back to ACTIVE because ₹250 is still unsettled
11	Aryan	**ROLLBACK:** create the failing trigger (below), accept Kabir's ₹350 bid, then drop the trigger	"Something went wrong". Balances stay ₹750 and ₹250, and the bid stays PENDING

```
mysql -u root -p campus_task_board -e "CREATE TRIGGER demo_fail BEFORE INSERT ON TRANSACTION_LOG FOR EACH ROW
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='forced failure'"
mysql -u root -p campus_task_board -e "DROP TRIGGER demo_fail"
```

To show the data directly: `SELECT name, balance, held_amount FROM USERS;` in MySQL, and `mongosh campus_task_board --eval "db.messages.find()"` for MongoDB. The README has the full 18-step test list.

7 Likely viva questions

> **HOW TO USE THE SHOW THIS COLUMN**
> If the teacher asks "where have you done this?", open the file and line in the right-hand column. In VS Code, press Ctrl+P, find the file, then press Ctrl+G and type the line number.

Written as	Means
`api/bids/[id]/accept:15`	Line 15 of `app/api/bids/[id]/accept/route.js`. Every `api/...` path is a `route.js` under `app/api/`
`sql:76`	Line 76 of `campus_task_board.sql`
`alter:63`	Line 63 of `alter_and_queries.sql`
Live:	Something to do in the running app, a terminal or the browser. "Demo step" means the table in section 6

Design

Question	Answer	Show this

How did you get a 1:1 relationship between BID and CONTRACT?	A foreign key alone gives 1:N. Making `bid_id` UNIQUE in CONTRACT limits each bid to one contract.	`sql:76`
		the UNIQUE column
		`api/bids/[id]/accept:53-56`
		where the contract row is created

Why is there no poster_id in CONTRACT?	It can be reached through CONTRACT → BID → TASK. Storing it again would be redundant and could go out of sync.	`sql:74-81`
		the table, with no poster column
		`lib/contracts.js:9-13`
		the joins that find poster and worker

Is the schema normalised?	Non-key columns depend on their table's key, and categories are in their own table. The deliberate exception is `avg_rating`, a stored derived value.	`sql:41-44, 57`
		category table, and TASK's foreign key to it
		`sql:27`
		the exception, marked as derived

Why store avg_rating if fn_avg_rating can calculate it?	It makes reads faster, and the trigger keeps it in sync. The dashboard shows both values, so you can see that they match.	`api/tasks/[id]:25`
		read directly for each bidder, with no recalculation
		`sql:279-286`
		the trigger
		`app/dashboard/page.js:53-54`
		both values side by side
		Live: Dashboard wallet, after demo step 8.

Why DECIMAL and not FLOAT for money?	DECIMAL stores exact values. FLOAT has rounding errors, which are not acceptable for money. If asked about `Number()` in the balance check: it only compares two values. The arithmetic itself runs in SQL.	`sql:28-29`
		balance and held_amount
		`api/bids/[id]/accept:37, 42`
		the JS comparison, then the SQL arithmetic
		`lib/api.js:24-31`
		input limited to 2 decimals, kept as text

DBMS concepts

Question	Answer	Show this

Why use a transaction?	One action changes several rows. Without it, a failure halfway could take money from the poster without creating the contract.	`api/bids/[id]/accept:15, 65, 75`
		start, commit, rollback
		`api/contracts/[id]/milestones:19`
		`api/milestones/[id]/approve:15`
		`api/milestones/[id]/dispute:22`
		`api/admin/disputes/[id]/refund:26`
		Live: demo step 11 forces a failure and nothing changes.

Question	Answer	Show this

Where can you show ROLLBACK working?	A temporary trigger makes the log insert fail. The route jumps to its catch block and rolls back, so the balance and bid changes made earlier in the same transaction are undone.	`api/bids/[id]/accept:59-63, 75`
		the insert that fails, and the rollback
		`alter:63, 68`
		SQL-only version: DELETE, then ROLLBACK
		Live: demo step 11.

What does FOR UPDATE do?	It locks the selected row until COMMIT or ROLLBACK, so a competing request has to wait, then sees the updated data.	`api/milestones/[id]/approve:19, 22`
		poster row, then milestone row
		`api/milestones/[id]/approve:31-33`
		where a double click is refused
		`api/bids/[id]/accept:20, 28`
		`api/admin/disputes/[id]/refund:28, 33`

How do you avoid deadlocks?	Every money route locks the poster's row first, so competing actions by the same poster queue up. This reduces deadlocks but cannot rule them out: if two students pay each other at the same moment, each transaction can hold its own row and need the other's. InnoDB detects this and rolls one back. That user gets an error and no money is lost.	`api/bids/[id]/accept:20`
		`api/contracts/[id]/milestones:22`
		`api/milestones/[id]/approve:19`
		`api/milestones/[id]/dispute:25`
		`api/admin/disputes/[id]/refund:28`
		`api/milestones/[id]/approve:39-42`
		the second row: the worker's
		`api/milestones/[id]/approve:62`
		undoes the transaction that InnoDB cancels

Which isolation level do you use?	MySQL's default, REPEATABLE READ. The project does not change it, and uses explicit row locks where money is involved.	`api/milestones/[id]/approve:15` no isolation level is set here or anywhere else in the code **Live:** run `SELECT @@transaction_isolation;`

Which constraints did you use, and where?	CHECK for value rules, UNIQUE for one-per-person rules, and foreign keys for valid references. The database enforces them even if the route code has a bug.	`sql:28-29, 53, 89`
		CHECK
		`sql:70, 94, 107`
		UNIQUE
		`sql:135-138`
		foreign keys on TRANSACTION_LOG

View vs table?	A view is a saved query with no data of its own. We use them to hide a join, and to expose only the wallet columns.	`sql:191-204, 206-208`
		definitions
		`api/dashboard:11, 38`
		where they are queried
		`app/dashboard/page.js:51-52, 59`
		where the results appear
		Live: Dashboard.

Function vs procedure?	A function returns one value and can be used inside a SELECT. A procedure is run with CALL, and can change data or return a result set.	`sql:221, 230`
		functions
		`api/tasks:29`
		function inside a SELECT
		`api/dashboard:15`
		`sql:252, 259`
		procedures
		`api/dashboard:13`
		returns a result set
		`lib/contracts.js:46`
		changes data
		Live: Days left column on Browse.

Why use triggers?	They run automatically on any change, whichever code made it, so the task status and the rating cannot be forgotten.	`sql:289-296, 279-286`
		definitions
		`api/bids/[id]/accept:46-47`
		fires here: note no UPDATE TASK in this route
		`api/contracts/[id]/review:25-28`
		fires here
		Live: demo steps 4 and 8.

Question	Answer	Show this

How does the database stop duplicate bids?	UNIQUE (task_id, bidder_id). The route catches ER_DUP_ENTRY and returns a clear 409 error.	`sql:70`
		the constraint
		`api/tasks/[id]/bids:39-41`
		the catch
		`app/tasks/[id]/page.js:51`
		the form also hides itself, for convenience
		Live: demo step 3.

Where did you use joins?	Everywhere a screen needs names as well as ids. The admin list is the largest: 7 tables, with USERS joined three times under different aliases. LEFT JOIN keeps rows whose optional match is missing, such as a task with no category.	`api/admin/disputes:13-21`
		7 tables, 3 aliases of USERS
		`api/tasks:33`
		category_id may be NULL
		`api/dashboard:29`
		not every bid has a contract

Where are aggregates, GROUP BY and subqueries?	The app uses them for bid counts, milestone totals and deciding when a contract is settled. The other query types are demonstrated in the SQL file.	`api/tasks:30`
		correlated subquery: bid count
		`api/contracts/[id]:43-47`
		SUM with GROUP BY
		`api/contracts/[id]/milestones:28-31`
		total check
		`lib/contracts.js:34-38`
		is the contract settled?
		`alter:92-99, 124, 134`
		HAVING, NOT IN, EXISTS

Did you create any indexes?	No separate CREATE INDEX. InnoDB automatically indexes every primary key, UNIQUE column and foreign key, and our lookups use those columns.	`sql:70, 76` these create indexes **Live:** run `SHOW INDEX FROM BID;`

MySQL and MongoDB

Question	Answer	Show this

Where is MongoDB actually used?	Only for the `messages` and `notifications` collections. They are simple documents that need no joins.	`lib/mongo.js:4-6, 10-12`
		connection, and both document shapes
		`api/contracts/[id]/messages:14-18, 35-41`
		read and write chat
		`lib/notify.js:7-14`
		write a notification
		`api/notifications:9-14, 25-27`
		list, and mark as read
		Live: `mongosh campus_task_board --eval "db.messages.find()"`

What if MongoDB is down during a payment?	The payment still succeeds. `notify()` catches the error and logs it, and only that notification is lost. Chat shows an error until MongoDB is back.	`lib/notify.js:15-17`
		the catch that protects the payment
		`lib/mongo.js:6`
		gives up after 3 seconds
		`components/ChatBox.js:18, 41`
		chat shows the error
		`components/Nav.js:20-23`
		unread count fails quietly

Can one transaction cover both databases?	No. So MySQL commits first and the MongoDB write comes after, and nobody is notified about a payment that was rolled back.	`api/bids/[id]/accept:65, 67`
		COMMIT, then notify
		`api/milestones/[id]/approve:52, 54`
		`api/admin/disputes/[id]/refund:61, 65`
		`lib/notify.js:3-4`
		the reason, in a comment

Is the chat real time?	It is polling: the browser asks every 3 seconds. This is simpler than WebSockets, and enough for this project.	`components/ChatBox.js:21-25`
		the timer, cleared on leaving the page
		`api/contracts/[id]/messages:14-18`
		the query each poll runs
		Live: demo step 6.

Question	Answer	Show this

How do the two databases connect? Who may read a chat?	MongoDB stores `contract_id` as a plain number, with no foreign key. Before every chat read or write, the route asks MySQL whether you are the contract's poster or worker.	`api/contracts/[id]/messages:12, 33`
		MySQL check before read and write
		`lib/contracts.js:22-28`
		the check itself
		`api/contracts/[id]/messages:36`
		the plain number stored in MongoDB

How does the unread notification count work?	Each document has a `read` flag. The nav bar counts the unread ones. Opening the notifications page marks them all as read.	`components/Nav.js:22, 45`
		count and badge
		`app/notifications/page.js:15`
		marks as read
		`api/notifications:25-27`
		the update

Did you index the collections?	No, only MongoDB's default `_id` index. That is fine for demo volumes. With real traffic we would index `contract_id` and `user_id`, the two fields we filter on.	`api/contracts/[id]/messages:16`
		filter on contract_id
		`api/notifications:11`
		filter on user_id
		Live: `mongosh campus_task_board --eval "db.messages.getIndexes()"`

Security

Question	Answer	Show this

How are passwords stored?	As bcrypt hashes with 10 salt rounds, never as plain text. bcrypt is one-way, so login compares against the hash.	`api/auth/register:20`
		hash at registration
		`api/auth/login:24`
		compare at login
		`alter:14-15`
		the columns
		`sql:206-208`
		wallet view leaves the hash out
		Live: `SELECT email, password_hash FROM USERS;`

How does login stay active?	A signed JWT holding id, name and role sits in an httpOnly cookie for 1 day. Changing it breaks the signature, so it is rejected.	`lib/session.js:11-16, 19-27`
		sign, and verify
		`lib/auth.js:24-32`
		the cookie settings
		`api/auth/login:28`
		set after a good login
		Live: DevTools → Application → Cookies shows HttpOnly. Edit the value and reload: you go to login.

Can someone register as an admin?	No. The register route always inserts the role STUDENT and ignores the form for this. Admins come from the ADMIN table.	`api/auth/register:7, 22-26`
		role is not read, and is fixed in the SQL
		`api/auth/login:12-21`
		role decided by which table has the email

Why check the role in proxy.js and again in the route?	Defence in depth. The proxy guards the paths, and the route needs the logged-in user's id anyway for its ownership checks.	`proxy.js:28-32`
		path check
		`lib/auth.js:10-22`
		route guards
		`api/admin/disputes/[id]/refund:12`
		route using one
		`api/milestones/[id]/approve:29`
		ownership check
		Live: log in as a student and open `/admin`: you go to `/tasks`.

Question	Answer	Show this

How do you stop one student acting on another's data?	Being logged in is not enough. Each route compares the logged-in id with the row's owner, and that id comes from the signed token, not from the request.	`lib/contracts.js:22-28`
		contract page and chat
		`api/bids/[id]/accept:33`
		accepting a bid
		`api/tasks/[id]:29-31`
		others' bids are not sent
		Live: log in as Kabir and open `/contracts/1`: "This is not your contract".

How do you prevent SQL injection?	All user input goes through `?` placeholders in prepared statements. Only fixed text is ever built into the SQL string.	`api/tasks:19, 26, 34`
		the one dynamic query
		`api/bids/[id]/accept:19-22`
		typical placeholder
		`api/bids/[id]/accept:15`
		`.query()` is only used for START TRANSACTION, COMMIT and ROLLBACK
		Live: search Browse for `' OR 1=1 --`: it is treated as text, so no rows match.

Does the login error reveal which emails exist?	No. A wrong email and a wrong password give the same message.	`api/auth/login:23-26` **Live:** demo step 1.

What about XSS and CSRF?	React escapes values placed in JSX, so a typed script shows as text, and the project never uses `dangerouslySetInnerHTML`. The httpOnly flag keeps page scripts away from the cookie. `sameSite: lax` means cross-site POST requests do not carry it.	`components/ChatBox.js:45`
		user text rendered safely
		`lib/auth.js:27, 28`
		cookie flags
		Live: send `<script>alert(1)</script>` in chat: it appears as plain text.

Where are the secrets kept?	In `.env.local`, which git ignores. `.env.example` is the template with no real values.	`.gitignore:4`
		ignored
		`lib/session.js:8`
		JWT secret
		`lib/db.js:8-12`
		database settings

What security is missing?	The cookie has no `secure` flag because the demo runs on http://localhost, and login has no rate limit. Both would be added before real deployment.	`lib/auth.js:26-31`
		cookie settings, with no secure line
		`api/auth/login:6-33`
		no attempt counter

Backend

Question	Answer	Show this
Where is your backend?	There is no separate server. Next.js runs the API in the same project. Each `route.js` under `app/api/` is an endpoint, and its exported GET or POST function handles that method.	`api/tasks:6, 48` both methods on one URL

Walk through one request.	`proxy.js` checks the cookie and role. The route checks the rules and runs the SQL inside a transaction. `ok()` returns JSON, and the page shows it. Next.js 16 calls this kind of file proxy; older versions called it middleware.	`proxy.js:7, 37-39`
		runs first, on every path
		`api/bids/[id]/accept:8-11, 73`
		the route, and its response
		`lib/api.js:11-13`
		`ok()`

How does the backend connect to MySQL?	One shared pool of up to 10 connections, so each request does not open its own. It is kept on `globalThis` so hot reload does not create new pools.	`lib/db.js:5-15, 17`

Why getConnection() in some routes and pool.execute() in others?	A transaction belongs to one connection, and `pool.execute()` may use a different one each call. So transaction routes take one connection, and `finally` always returns it to the pool.	`api/bids/[id]/accept:14, 78`
		take and return
		`api/tasks/[id]:11`
		simple read, no transaction needed

Question	Answer	Show this
Why does submit not use a transaction?	It is one UPDATE, and a single statement is already atomic. Its WHERE clause holds all the checks, and `affectedRows` shows whether they passed.	`api/milestones/[id]/submit:13-21, 22-24`

Where do you validate input?	In the routes, before touching the database, since a request can skip the browser form entirely. CHECK constraints are the last line of defence.	`lib/api.js:25-31`
		shared amount check
		`api/tasks:53-61`
		task fields
		`api/auth/register:10-14`
		registration
		`api/contracts/[id]/review:14-17`
		review

How are errors handled?	Routes throw `ApiError` with a status and a message for the user. `handleError` sends it as JSON. Unexpected errors are logged, and the user sees only "Something went wrong", so no SQL details leak.	`lib/api.js:4-9, 16-22`
		`api/bids/[id]/accept:32, 33, 35, 38`
		404, 403, 409, 400 in one route
		`proxy.js:19`
		401 when logged out

How is the browse filter built?	It starts with `status = 'OPEN'`, and each filter that was filled in adds a condition with `?` and a value. The conditions are then joined with AND.	`api/tasks:10-24, 34` **Live:** use the filters on Browse.

Where is the rule for closing a contract?	It adds up RELEASE and REFUND rows. If they do not reach the agreed amount, the contract is ACTIVE. If they do, it is COMPLETED when anything was released, and otherwise REFUNDED.	`lib/contracts.js:33-52`
		the rule
		`api/milestones/[id]/approve:50`
		`api/admin/disputes/[id]/refund:59`
		Live: demo steps 7 and 10.

How do you reset the demo data?	`npm run reset` drops the MySQL database, runs both SQL files, then drops the MongoDB database.	`package.json:9` `scripts/reset-db.mjs:37-39, 42`

Frontend

Question	Answer	Show this

Where is your frontend? How do pages map to URLs?	React 19 pages inside Next.js. The folder path is the URL, so `app/tasks/[id]/page.js` serves `/tasks/7`, and `[id]` is read with `useParams`. `layout.js` wraps every page with the nav bar and fonts, and `globals.css` holds the styling.	`app/tasks/[id]/page.js:9`
		reads the id from the URL
		`app/layout.js:13-22`
		shared shell
		`app/globals.css`
		all the styling

How does a page talk to the backend?	Through one helper, `api()`, which wraps `fetch`. It sends JSON, and turns any error response into a thrown error for the page to show.	`lib/client.js:2-14`
		the helper
		`app/tasks/[id]/page.js:41`
		the Accept button's call
		`app/dashboard/page.js:37`
		loading a page

Why do the pages start with "use client"?	The pages use hooks and event handlers, which run in the browser. They render a loading state, fetch JSON from the API, then show it. No page queries a database directly.	`app/dashboard/page.js:1, 36-38, 40` directive, fetch, loading state
How does the screen update after an action?	Data lives in `useState`. After an action succeeds, `run()` calls `load()` to fetch again, so the screen always shows what the database now holds.	`app/contracts/[id]/page.js:10, 18-24, 27-39` state, loading, and the action helper **Live:** Approve a milestone, and the status and log update.
How do your forms work?	Each input's value comes from state, and `onChange` updates it. On submit, `preventDefault` stops the page reload, then `api()` sends the state as JSON.	`app/login/page.js:8, 11-20, 27-28`

Question	Answer	Show this
Is browser validation enough?	No. The browser check gives quick feedback, but anyone can call the API directly, so the server repeats every rule.	`app/contracts/[id]/page.js:168` browser limit `api/contracts/[id]/milestones:34-36` same rule on the server **Live:** demo step 5.
How do poster and worker see different buttons?	The API returns the user's role on the contract, and the page shows buttons by role and status. This is only for usability. The route still checks permission on every request.	`api/contracts/[id]:58` API sends the role `app/contracts/[id]/page.js:103, 109` Submit, Approve `api/milestones/[id]/approve:29` the real check **Live:** same contract in two windows.
How does the nav bar change by role?	It calls `/api/auth/me` when the path changes. Students get task, dashboard and notification links with an unread count. Admins get only Disputes.	`components/Nav.js:16, 26, 39-50`
What happens if the session expires?	The API returns 401, and `api()` sends the browser to the login page. Opening a page while logged out is redirected by the proxy.	`lib/client.js:9-11` `proxy.js:20`
How are errors shown to the user?	`api()` throws the server's message. Each page catches it, stores it in state and displays it, so the rule is written once, on the server.	`lib/client.js:12` throws `app/tasks/page.js:19-21, 57` catches, and displays **Live:** accept a bid with too little balance.
Where is money formatted?	`money()` adds ₹ and 2 decimals for display only. It never changes stored values.	`lib/client.js:16-18` `app/dashboard/page.js:51-52`

8 Limitations: know these before they are pointed out

It is better to raise these yourself, and each one has a short honest answer.

Limitation	What to say
The admin can only refund. DISPUTE has a REJECTED status, but no route uses it	If the worker was in the right, the contract stays frozen. A "reject dispute" route is the first thing we would add
`resolved_by` **is always NULL**	It references USERS, but admins are in ADMIN, so `admin_id` records who handled the dispute. The column could be removed
A refunded milestone stays FLAGGED	The milestone ENUM has no refunded status. The dispute row and the REFUND log entry show the outcome
No top-up or withdrawal. New students start at ₹0	The money is simulated, so wallets are out of scope. A new student can bid and earn, but cannot accept a bid until they have earned enough
No way to cancel a task or contract	Money stays held until each milestone is approved or refunded, so milestones must be added to use it all
Deadlines are displayed, not enforced	An overdue task stays OPEN and shows negative days left
`USERS.role` **is never ADMIN**	It came from the original design. Login decides the role by which table the email is in

Check these in your report's diagrams

The relational schema on page 3 of the report may draw questions. Several foreign key arrows land on the wrong column (CATEGORY points to USER, and BID points to TASK's `poster_id` and `category_id`), and `admin_id` is underlined in DISPUTE as if it were part of the primary key. The copy in the repo, `Relational Schema.png`, also has the heading "Freelance Platform". The foreign keys actually in the database are:

Table	Foreign keys in campus_task_board.sql
TASK	`poster_id` → USERS, `category_id` → CATEGORY
BID	`task_id` → TASK, `bidder_id` → USERS
CONTRACT	`bid_id` → BID
MILESTONE	`contract_id` → CONTRACT
REVIEW	`contract_id` → CONTRACT, `reviewer_id` → USERS, `reviewee_id` → USERS
DISPUTE	`milestone_id` → MILESTONE, `raised_by` → USERS, `resolved_by` → USERS, `admin_id` → ADMIN
TRANSACTION_LOG	`contract_id` → CONTRACT, `milestone_id` → MILESTONE, `from_user` → USERS, `to_user` → USERS